Test Strategy – Mobile Application (Android & iOS) – End-to-End Role-Based Access Validation
(Prepared for the "Web UI Role-Based Access Control" effort – now extended to the companion mobile client)

---

1. Objective

Validate that the **mobile client** (Android & iOS) enforces the same role-based permissions as the web UI:

Role	Mobile-only Capabilities	Web-UI Equivalent (for reference)
Admin	• Manage *Parent-Control*, *Firewall*, *ACL* • View *System-wide* logs	Settings → Parent-Control, Firewall, ACL
Engineer	• Upload firmware • Edit ISP configuration • Run diagnostics	Engineer → Firmware, ISP Config, Diagnostics

The tests must prove that **only the correct role can see & execute each feature** on both platforms, and that **unauthorized attempts are blocked** (UI-level, API-level, and audit-log level).

---

2. Test Scope

In-Scope	Out-Of-Scope
• Login flow for Admin & Engineer on Android12+ and iOS17+ • Navigation to all role-specific screens • Positive actions (create, edit, delete) • Negative actions (direct deep-link, API call, UI manipulation) • Session timeout & logout handling	• Native push-notification handling • Device-level security (root/jailbreak detection) • Performance / battery consumption • Accessibility compliance
• Cross-platform data sync verification (mobile web)	• Third-party SDK behaviour (e.g., analytics)

---

3. Test Levels & Types

Test Level	Test Type	Tools	Goal
Component	UI functional tests (Appium, Espresso, XCUITest)	Appium2.0 (cross-platform), Espresso (Android), XCUITest (iOS)	Verify each screen respects role permissions
System	End-to-End workflow (Cypress-Mobile Bridge or Detox)	Detox (React-Native) or Cypress+Appium bridge	Validate full user journey (login → config change → sync)
Security	Access-control penetration (OWASPMobile-ZAP, Burp Suite)	Mobile-ZAP, Burp Suite Pro	Ensure no privilege escalation via intercepted traffic, deep links, or malformed intents
Acceptance	Role-Based Acceptance Test (RB-AAT)	Cucumber (Gherkin) + Appium	Business-level sign-off that Admin & Engineer can only do their allowed actions

---

4. Test Design Approach

1.

Requirement Traceability Matrix (RTM) *– Map every functional requirement (e.g., "Admin can enable Parent-Control") to one or more mobile test cases.*

2.
3.

Equivalence Partitioning *– Two partitions per feature: Authorized (role has permission) vs. Unauthorized (role does* **not** *have permission).*

4.

5.

Data-Driven Tests – Use external CSV/JSON files containing:

6.
7.

```
[  { "role":"admin",   "username":"admin1",
"password":"P@ssw0rd!" },  { "role":"engineer","username":"eng1",
"password":"Eng!2026" }]
```

8.
9.

The same test script loops over the data set, injecting the credentials at runtime.

10.
11.

Deep-Link Negative Tests – Attempt to launch a prohibited screen via a custom URI scheme (e.g.,

12.

```
myapp://firewall
```

13.) while logged in as Engineer.
14.
15.

API-Level Validation – Intercept the mobile app's REST calls (via a proxy) and attempt the same request with an Engineer token against an Admin-only endpoint; verify HTTP 403.

16.
17.

Audit-Log Verification – After each successful/failed operation, query the backend audit service (REST) to confirm the log entry reflects the correct user, role, and outcome.

18.

---

5. Test Environment

Component	Version	Configuration
Device Farm	Azure App Center Device Lab (real devices)	Android 12 (Pixel 6), Android 13 (Galaxy S23), iOS 17 (iPhone 14)
Mobile App Build	`myapp-android-3.4.2.apk` , `myapp-ios-3.4.2.ipa`	Signed with QA certificate, feature-flags ON for all role screens
Backend (Staging)	API v2.9 (HTTPS, JWT auth)	URL: `https://staging-api.example.com/v2`
Proxy for Traffic Capture	OWASP Mobile-ZAP (running on a dedicated VM)	MITM certificate installed on devices
Test Automation Hub	Jenkins 2.462 + GitLab CI	Parallel execution on 6 Android & 4 iOS devices
Test Data Store	Azure Blob (`test-data/`)	Version-controlled, encrypted at rest

*All devices are **factory-reset** before each test run, and the mobile app is freshly installed.*

---

6. Test Data

Data Set	Description	Example

User Credentials	Admin & Engineer accounts (pre-seeded in staging DB)	`admin1 / P@ssw0rd!`, `eng1 / Eng!2026`
Firmware File	Small dummy binary (≈1 MB)	`dummy_firmware_v1.2.bin`
Parent-Control Rule	JSON rule used for creation test	`{ "block": ["social.com"], "schedule":"22:00-06:00" }`
ISP Config JSON	Payload for ISP edit	`{ "dns":"1.1.1.1", "gateway":"192.168.1.1" }`
Deep-Link URIs	Custom scheme for each screen	`myapp://parent-control`, `myapp://firmware`
Invalid Token	JWT with wrong role claim	`{ "sub":"eng1","role":"engineer","exp":... }` (signed with wrong key)

All files reside in

```
test-data/mobile/
```

and are referenced by the automation scripts.

---

7. Test Cases (Representative Matrix)

Notation —

`TC-M-<role>-<seq>`

(M = Mobile). Full suite (≈ 180 cases) is in

`Mobile_RBAC_TestCases.xlsx`

.

TC-ID	Platform	Role	Feature	Test Steps (high-level)	Expecte
TC-M-ADMIN-001	Android & iOS	Admin	**Login**	1. Launch app 2. Enter admin credentials 3. Tap **Sign In**	Dashboard *Admin* bad token inc `"role":"ad`
TC-M-ADMIN-010	Android	Admin	**Parent-Control - Add Rule**	1. Open **Settings → Parent-Control** 2. Tap **+ Add** 3. Fill rule JSON (from test data) 4. Save	New rule list, suc “Rule ad backend D created
TC-M-ADMIN-020	iOS	Admin	**Firewall - Edit**	1. Navigate to **Firewall** 2. Choose existing rule 3. Change port from 8080 → 9090 4. Save	Rule upda “Saved” returns `200` with new
TC-M-ADMIN-030	Android	Admin	**Engineer-Only - Firmware Upload (Negative)**	1. Directly open deep-link `myapp://firmware` 2. Observe screen	Access-de “Insuffi permissio upload UI
TC-M-ENG-001	Android & iOS	Engineer	**Login**	Same as TC-M-ADMIN-001 but Engineer credentials	Dashboard *Engineer* claim

					`"role":"en`
TC-M-ENG-010	iOS	Engineer	Firmware Upload	1. Open Engineer → Firmware 2. Tap Upload 3. Select `dummy_firmware_v1.2.bin` 4. Confirm	